

R4DS 10 – Factors, Dates & Missing Values

Factor Basics

Why Factors?

Factors handle **categorical variables** with a fixed set of possible values.

Two problems with plain strings:

```
x1 <- c("Dec", "Apr", "Jan", "Mar")  
sort(x1) # alphabetical, not calendar order
```

```
#> [1] "Apr" "Dec" "Jan" "Mar"
```

- ❶ **No typo protection** — any string is accepted
- ❷ **No meaningful order** — sorts alphabetically

Creating Factors

Define valid **levels**, then create:

```
month_levels <- c("Jan", "Feb", "Mar", "Apr",  
  "May", "Jun", "Jul", "Aug",  
  "Sep", "Oct", "Nov", "Dec")  
y1 <- factor(x1, levels = month_levels)  
y1
```

```
#> [1] Dec Apr Jan Mar  
#> 12 Levels: Jan Feb Mar Apr May Jun Jul Aug Sep ... Dec
```

```
sort(y1)
```

```
#> [1] Jan Mar Apr Dec  
#> 12 Levels: Jan Feb Mar Apr May Jun Jul Aug Sep ... Dec
```

Invalid values become **NA** silently. Use **fct()** for strict checking:

```
fct(c("Dec", "Apr", "Jam"), levels = month_levels)
```

```
#> Error in `fct()`:  
#> ! All values of `x` must appear in `levels` or  
#>   `na`  
#> i Missing level: "Jam"
```

The gss_cat Dataset

`forcats::gss_cat` — sample from the General Social Survey:

```
gss_cat
```

```
#> # A tibble: 21,483 x 9
#>   year marital      age race  rincome partyid relig
#>   <int> <fct>      <int> <fct> <fct>   <fct>   <fct>
#> 1  2000 Never marri~    26 White $8000 ~ Ind,ne~ Prot~
#> 2  2000 Divorced      48 White $8000 ~ Not st~ Prot~
#> 3  2000 Widowed      67 White Not ap~ Indepe~ Prot~
#> 4  2000 Never marri~    39 White Not ap~ Ind,ne~ Orth~
#> 5  2000 Divorced      25 White Not ap~ Not st~ None
#> ... [output truncated]
```

View levels with `count()`:

```
gss_cat |> count(race)
```

```
#> # A tibble: 3 x 2
#>   race      n
#>   <fct> <int>
#> 1 Other  1959
#> 2 Black  3129
#> 3 White 16395
```

Exercises (Factor Basics)

- 1 Explore the distribution of `rincome` (reported income). What makes the default bar chart hard to understand? How to improve it?
- 2 What is the most common `relig`? Most common `partyid`?
- 3 Which `relig` does `denom` (denomination) apply to? How can you find out with a table? With a visualization?

Solutions (Factor Basics)

- 1 Labels overlap and the order is counterintuitive (income categories are reversed). Improve: reorder levels or use `coord_flip()`.

2

```
gss_cat |> count(relig, sort = TRUE) |> head(1)
```

```
#> # A tibble: 1 x 2  
#>   relig      n  
#>   <fct>    <int>  
#> ... [output truncated]
```

```
gss_cat |> count(partyid, sort = TRUE) |> head(1)
```

```
#> # A tibble: 1 x 2  
#>   partyid      n  
#>   <fct>    <int>  
#> ... [output truncated]
```

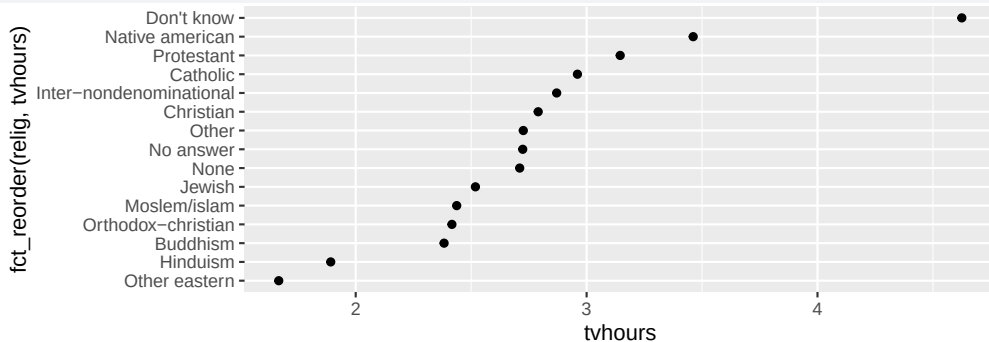
- 3 Mainly Protestant. Check: `gss_cat |> count(relig, denom)` or `geom_count(aes(x = relig, y = denom))`.

Modifying Factor Order

fct_reorder() — Reorder by Another Variable

```
relig_summary <- gss_cat |>
  group_by(relig) |>
  summarize(tvhours = mean(tvhours, na.rm = TRUE),
            n = n())

ggplot(relig_summary,
       aes(x = tvhours,
           y = fct_reorder(relig, tvhours))) +
  geom_point()
```



fct_relevel() and fct_reorder2()

`fct_relevel()` moves specific levels to front (for principled orderings):

```
ggplot(rincome_summary,  
  aes(x = age,  
      y = fct_relevel(rincome, "Not applicable"))) +  
  geom_point()
```

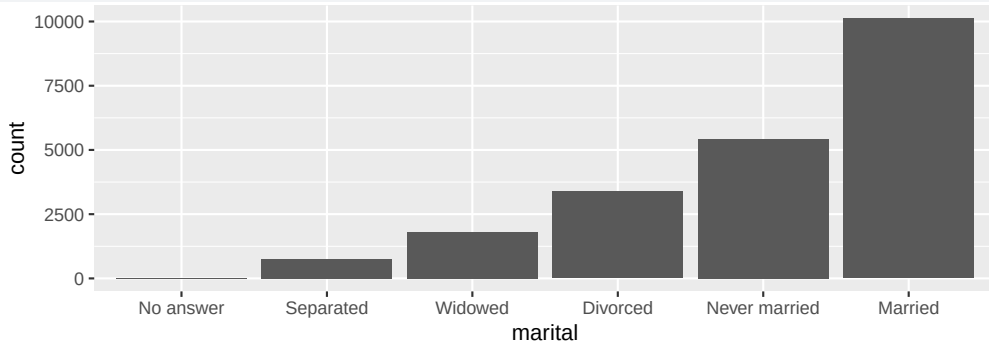
`fct_reorder2(.f, .x, .y)` — reorders by `.y` at largest `.x`. Legend aligns with line endpoints:

```
ggplot(by_age,  
  aes(x = age, y = prop,  
      color = fct_reorder2(marital, age, prop))) +  
  geom_line() + labs(color = "marital")
```

fct_infreq() and fct_rev()

Order by **frequency** — simplest reordering (no extra variable):

```
gss_cat |>
  mutate(marital = marital |>
    fct_infreq() |> fct_rev()) |>
  ggplot(aes(x = marital)) +
  geom_bar()
```



`fct_infreq()` orders most-to-least; `fct_rev()` reverses.

Exercises (Factor Order)

- 1 There are suspiciously high numbers in `tvhours`. Is the mean a good summary?
- 2 For each factor in `gss_cat`, identify whether the order of levels is arbitrary or principled.
- 3 Why did moving “Not applicable” to the front of levels move it to the **bottom** of the plot?

Solutions (Factor Order)

- ❶ No — outliers skew the mean. Use median, or filter extremes first.
- ❷ `marital`, `race`, `relig`, `denom`, `partyid` are arbitrary. `rincome` is principled (ordered income bands).
- ❸ Because `ggplot2` plots factor levels from bottom to top on the y-axis. The first level appears at the bottom.

Modifying Factor Levels

fct_recode() — Rename Levels

New values on left, old on right:

```
gss_cat |>
  mutate(partyid = fct_recode(partyid,
    "Republican, strong" = "Strong republican",
    "Republican, weak"   = "Not str republican",
    "Independent, near rep" = "Ind,near rep",
    "Independent, near dem" = "Ind,near dem",
    "Democrat, weak"      = "Not str democrat",
    "Democrat, strong"    = "Strong democrat"
  )) |> count(partyid)
```

```
#> # A tibble: 10 x 2
```

```
#>   partyid      n
#>   <fct>    <int>
#> 1 No answer    154
#> 2 Don't know     1
#> 3 Other party   393
#> 4 Republican, strong 2314
#> 5 Republican, weak  3032
#> ... [output truncated]
```

fct_collapse() and fct_lump_*()

fct_collapse() groups multiple levels at once:

```
gss_cat |>
  mutate(partyid = fct_collapse(partyid,
    "other" = c("No answer", "Don't know",
      "Other party"),
    "rep" = c("Strong republican",
      "Not str republican"),
    "ind" = c("Ind,near rep", "Independent",
      "Ind,near dem"),
    "dem" = c("Not str democrat",
      "Strong democrat")
  )) |> count(partyid)
```

```
#> # A tibble: 4 x 2
#>   partyid     n
#>   <fct>   <int>
#> 1 other     548
#> 2 rep     5346
#> 3 ind     8409
#> 4 dem     7180
#> ... [output truncated]
```



```
gss_cat |>
  mutate(relig = fct_lump_n(relig, n = 5)) |>
  count(relig, sort = TRUE)
```

```
#> # A tibble: 6 x 2
#>   relig      n
#>   <fct>    <int>
#> 1 Protestant 10846
#> 2 Catholic   5124
#> 3 None       3523
#> 4 Other       913
#> 5 Christian   689
#> ... [output truncated]
```

Family: `fct_lump_n()` (keep top n), `fct_lump_min()` (min count), `fct_lump_prop()` (min proportion), `fct_lump_lowfreq()` (keep “Other” smallest).

Exercises (Factor Levels)

- ❶ How have proportions of people identifying as Democrat, Republican, and Independent changed over time?
- ❷ How could you collapse `rincome` into a small set of categories?
- ❸ In the `fct_lump` example, why 9 groups (not 10) besides “Other”?

Solutions (Factor Levels)

1

```
gss_cat |>
  mutate(partyid = fct_collapse(partyid,
    "rep" = c("Strong republican",
      "Not str republican"),
    "dem" = c("Not str democrat",
      "Strong democrat"),
    "ind" = c("Ind,near rep", "Independent",
      "Ind,near dem"),
    "other" = c("No answer", "Don't know",
      "Other party"))) |>
  count(year, partyid) |>
  group_by(year) |> mutate(prop = n / sum(n)) |>
  ggplot(aes(year, prop, color = partyid)) +
  geom_line()
```

- ② Group adjacent income bands:

```
gss_cat |>
  mutate(rincome = fct_collapse(rincome,
    "Unknown" = c("No answer", "Don't know",
      "Refused", "Not applicable"),
    "< $5000" = c("Lt $1000", "$1000 to 2999",
      "$3000 to 3999",
      "$4000 to 4999"),
    "$5000-9999" = c("$5000 to 5999",
      "$6000 to 6999", "$7000 to 7999",
      "$8000 to 9999"),
    "$10000+" = c("$10000 - 14999",
      "$15000 - 19999", "$20000 - 24999",
      "$25000 or more")))
```

- ③ “Other” counts as one of the groups internally. With `n = 10`, you get 10 groups total: 9 named + “Other”.

Creating Dates and Times

Three Types

Type	Tibble prints	Example
Date	<date>	2024-03-31
Time	<time>	12:30:00 (use hms package)
Date-time	<dtm>	2024-03-31 12:30:00

Use the **simplest** type that works. Dates are simpler than date-times (no time zone issues).

```
today()
```

```
#> [1] "2026-04-01"
```

```
now()
```

```
#> [1] "2026-04-01 16:39:13 CST"
```

From Strings: ymd(), mdy(), dmy()

Arrange “y”, “m”, “d” in the order they appear:

```
ymd("2017-01-31")
```

```
#> [1] "2017-01-31"
```

```
mdy("January 31st, 2017")
```

```
#> [1] "2017-01-31"
```

```
dmy("31-Jan-2017")
```

```
#> [1] "2017-01-31"
```

Add time components with underscore:

```
ymd_hms("2017-01-31 20:11:59")
```

```
#> [1] "2017-01-31 20:11:59 UTC"
```

```
mdy_hm("01/31/2017 08:01")
```

```
#> [1] "2017-01-31 08:01:00 UTC"
```

From Components: make_date() / make_datetime()

When date parts are in separate columns:

```
flights |>
  select(year, month, day, hour, minute) |>
  mutate(departure = make_datetime(
    year, month, day, hour, minute))

#> # A tibble: 336,776 x 6
#>   year month   day hour minute departure
#>   <int> <int> <int> <dbl> <dbl> <dtm>
#> 1  2013     1     1     5     15 2013-01-01 05:15:00
#> 2  2013     1     1     5     29 2013-01-01 05:29:00
#> 3  2013     1     1     5     40 2013-01-01 05:40:00
#> ... [output truncated]
```


Exercises (Creating Dates)

- 1 What happens if you parse an invalid date?

```
ymd(c("2010-10-10", "bananas"))
```

- 2 What does the `tzzone` argument to `today()` do? Why is it important?
- 3 Parse each of these:

```
d1 <- "January 1, 2010"  
d2 <- "2015-Mar-07"  
d3 <- "06-Jun-2017"  
d4 <- c("August 19 (2015)", "July 1 (2015)")  
d5 <- "12/30/14"
```

Solutions (Creating Dates)

- 1 Returns **NA** with a warning for the invalid string.
- 2 **tzzone** sets the time zone used to determine the date. Important because “today” depends on your location — it can be different dates in different time zones.
- 3

```
mdy("January 1, 2010")
```

```
#> [1] "2010-01-01"
```

```
ymd("2015-Mar-07")
```

```
#> [1] "2015-03-07"
```

```
dmy("06-Jun-2017")
```

```
#> [1] "2017-06-06"
```

```
mdy(c("August 19 (2015)", "July 1 (2015)"))
```

```
#> [1] "2015-08-19" "2015-07-01"
```

```
mdy("12/30/14")
```

```
#> [1] "2014-12-30"
```

Date-Time Components

Getting Components

Accessor functions extract parts of a date-time:

```
datetime <- ymd_hms("2026-07-08 12:34:56")  
year(datetime)
```

```
#> [1] 2026
```

```
month(datetime)
```

```
#> [1] 7
```

```
mday(datetime)
```

```
#> [1] 8
```

```
yday(datetime)
```

```
#> [1] 189
```

```
wday(datetime)
```

```
#> [1] 4
```

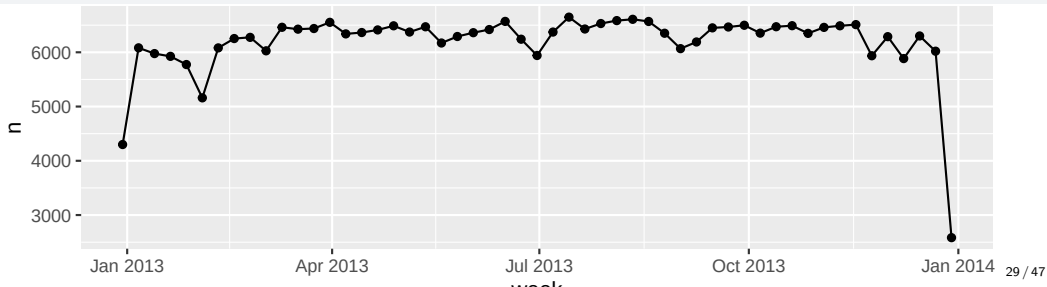
`label = TRUE` returns names: `month(datetime, label = TRUE)`.

Rounding Dates

`floor_date()`, `round_date()`, `ceiling_date()` round to a time unit:

```
flights_dt <- flights |>
  filter(!is.na(dep_time), !is.na(arr_time)) |>
  mutate(dep_time = make_datetime(
    year, month, day,
    dep_time %% 100, dep_time %% 100))
```

```
flights_dt |>
  count(week = floor_date(dep_time, "week")) |>
  ggplot(aes(x = week, y = n)) +
  geom_line() + geom_point()
```



Time Spans

Durations — Exact Seconds

Subtracting dates gives a `difftime`. Lubridate's **durations** always use seconds:

```
dseconds(15)
```

```
#> [1] "15s"
```

```
dminutes(10)
```

```
#> [1] "600s (~10 minutes)"
```

```
dhours(c(12, 24))
```

```
#> [1] "43200s (~12 hours)" "86400s (~1 days)"
```

```
ddays(0:3)
```

```
#> [1] "0s" "86400s (~1 days)"
```

```
#> [3] "172800s (~2 days)" "259200s (~3 days)"
```

```
dyears(1)
```

```
#> [1] "31557600s (~1 years)"
```

Durations are exact: `dyears(1)` = 365.25 days in seconds.

Periods work with “human” time (days, months) without fixed second count:

```
days(7)
```

```
#> [1] "7d 0H 0M 0S"
```

```
months(1:6)
```

```
#> [1] "1m 0d 0H 0M 0S" "2m 0d 0H 0M 0S" "3m 0d 0H 0M 0S"
```

```
#> [4] "4m 0d 0H 0M 0S" "5m 0d 0H 0M 0S" "6m 0d 0H 0M 0S"
```

```
10 * (months(6) + days(1))
```

```
#> [1] "60m 10d 0H 0M 0S"
```


Durations vs. Periods: DST

Durations add exact seconds; periods respect calendar rules:

```
one_am <- ymd_hms("2026-03-08 01:00:00",  
                  tz = "America/New_York")  
one_am + ddays(1)    # 2am! (DST = 23-hr day)
```

```
#> [1] "2026-03-09 02:00:00 EDT"
```

```
one_am + days(1)      # 1am (as expected)
```

```
#> [1] "2026-03-09 01:00:00 EDT"
```

Leap years:

```
ymd("2024-01-01") + dyears(1) # Dec 31!
```

```
#> [1] "2024-12-31 06:00:00 UTC"
```

```
ymd("2024-01-01") + years(1)  # Jan 1, 2025
```

```
#> [1] "2025-01-01"
```

Intervals — Duration with Start Point

`start %--% end` creates an interval for precise calculations:

```
y2023 <- ymd("2023-01-01") %--% ymd("2024-01-01")  
y2024 <- ymd("2024-01-01") %--% ymd("2025-01-01")  
y2023 / days(1)
```

```
#> [1] 365
```

```
y2024 / days(1)
```

```
#> [1] 366
```

Rule of thumb: physical time → duration, human time → period, how long in human units → interval.

Exercises (Time Spans)

- 1 Explain `days(overnight)` to a beginner. What key fact about R makes it work?
- 2 Create a vector of dates for the first day of every month in 2015.
- 3 Write a function that, given your birthday (as a date), returns your age in years.

Solutions (Time Spans)

- ① `overnight` is logical (`TRUE/FALSE`). R coerces `TRUE` to 1 and `FALSE` to 0, so `days(TRUE) = 1` day, `days(FALSE) = 0` days.

②

```
ymd("2015-01-01") + months(0:11)
```

```
#> [1] "2015-01-01" "2015-02-01" "2015-03-01"  
#> [4] "2015-04-01" "2015-05-01" "2015-06-01"  
#> [7] "2015-07-01" "2015-08-01" "2015-09-01"  
#> ... [output truncated]
```

③

```
age <- function(bday) {  
  (bday %--% today()) %/% years(1)  
}  
age(ymd("1990-05-15"))
```

```
#> [1] 35
```

Time Zones

R uses IANA time zones: "America/New_York", "Europe/Paris", "Pacific/Auckland".

Two ways to change time zone:

Function	What it does
<code>with_tz()</code>	Same instant, different display
<code>force_tz()</code>	Changes the instant (fix mislabeled tz)

```
x <- ymd_hms("2024-06-01 12:00:00",  
            tz = "America/New_York")  
with_tz(x, "Europe/Paris")
```

```
#> [1] "2024-06-01 18:00:00 CEST"
```

Explicit Missing Values

fill() and coalesce()

`fill()` carries the last observation forward:

```
treatment <- tribble(
  ~person,      ~treatment, ~response,
  "Derrick Whitmore", 1,      7,
  NA,            2,      10,
  NA,            3,      NA,
  "Katherine Burke", 1,      4)
treatment |> fill(everything())
```

```
#> # A tibble: 4 x 3
```

```
#>   person      treatment response
#>   <chr>      <dbl>     <dbl>
#> 1 Derrick Whitmore      1         7
#> 2 Derrick Whitmore      2        10
#> 3 Derrick Whitmore      3        10
#> 4 Katherine Burke       1         4
```

coalesce() and na_if()

Replace NA with a fixed value:

```
x <- c(1, 4, 5, 7, NA)
coalesce(x, 0)
```

```
#> [1] 1 4 5 7 0
```

Replace a sentinel value with NA:

```
x <- c(1, 4, 5, 7, -99)
na_if(x, -99)
```

```
#> [1] 1 4 5 7 NA
```

NaN (“not a number”) — behaves like NA. Arises from $0/0$, $\text{Inf} - \text{Inf}$, $\text{sqrt}(-1)$. Detect with `is.nan()`.

Implicit Missing Values

```
stocks <- tibble(  
  year = c(2020, 2020, 2020, 2020,  
           2021, 2021, 2021),  
  qtr  = c(1, 2, 3, 4, 2, 3, 4),  
  price = c(1.88, 0.59, 0.35, NA,  
            0.92, 0.17, 2.66))
```

- **Explicit:** 2020 Q4 price is **NA** — the presence of an absence
- **Implicit:** 2021 Q1 is missing entirely — the absence of a presence

```
stocks |> complete(year, qtr)
```

```
#> # A tibble: 8 x 3  
#>   year    qtr price  
#>   <dbl> <dbl> <dbl>  
#> 1  2020     1  1.88  
#> 2  2020     2  0.59  
#> 3  2020     3  0.35  
#> 4  2020     4  NA  
#> 5  2021     1  NA  
#> 6  2021     2  0.92  
#> 7  2021     3  0.17  
#> ... [output truncated]
```

Generates all combinations of variables, filling gaps with **NA**.

Can supply ranges: `complete(year = 2019:2021, qtr)`.

pivot_wider() and Joins

`pivot_wider()` also reveals implicit missings:

```
stocks |>
  pivot_wider(names_from = qtr,
              values_from = price)
```

```
#> # A tibble: 2 x 5
#>   year   `1`   `2`   `3`   `4`
#>   <dbl> <dbl> <dbl> <dbl> <dbl>
#> 1  2020  1.88  0.59  0.35 NA
#> 2  2021 NA    0.92  0.17  2.66
#> ... [output truncated]
```

`anti_join()` finds rows in one table with no match in another — useful for detecting missing data across datasets:

```
flights |> distinct(faa = dest) |>
  anti_join(airports)
```

Factors and Empty Groups

Factor levels with zero observations are dropped by default:

```
health <- tibble(  
  name = c("Ikaia", "Oletta", "Leriah"),  
  smoker = factor(c("no", "no", "no"),  
                 levels = c("yes", "no"))  
health |> count(smoker)
```

```
#> # A tibble: 1 x 2  
#>   smoker      n  
#>   <fct>   <int>  
#> 1 no           3
```

Keep all levels with `.drop = FALSE`:

```
health |> count(smoker, .drop = FALSE)
```

```
#> # A tibble: 2 x 2  
#>   smoker      n  
#>   <fct>   <int>  
#> 1 yes         0  
#> 2 no          3  
#> ... [output truncated]
```

Same for `group_by(.drop = FALSE)` and `ggplot2 scale_x_discrete(drop = FALSE)`.

Exercises (Missing Values)

- ❶ What happens to missing values in a histogram vs. a bar chart?
- ❷ What do `mean()` and `sum()` return for a zero-length vector vs. a vector with `NA`?
- ❸ Can you find a relationship between `carrier` and the rows missing from `planes`?

Solutions (Missing Values)

❶ Histograms silently drop NAs (with a warning). Bar charts show NAs as a separate category.

❷

```
mean(c(1, NA))          # NA (infectious)
```

```
#> [1] NA
```

```
mean(c(1, NA), na.rm = TRUE) # 1
```

```
#> [1] 1
```

```
mean(numeric())         # NaN (0/0)
```

```
#> [1] NaN
```

```
sum(numeric())          # 0
```

```
#> [1] 0
```

❸ MQ and AA have many planes not in the `planes` table — they report tail numbers differently.